

Finding an Optimal Path for the Elderly Considering the Costs Based on Skyline to Maximize User Preference

Ria Sunwoo An

North London Collegiate School, Jeju, Republic of Korea

ria.an2027@gmail.com

Abstract

In modern society, navigation systems enable human beings to find the optimal route quickly. Although it has been reported that Korea will be classified as a "super-aged" society by 2025 and it is the country with the highest elderly pedestrian death rate, there is no proper navigation system suited for the elderly. While the majority of navigation systems have decreasing time as their priority, the important thing for seniors is safety. In this research, I studied the criteria and the factors that the navigation system should consider for the elderly when finding the safest path and defining score functions based on it. The brute force and greedy algorithms are introduced as solutions for this newly defined score function. By adopting the Skyline Query, the performance and utility of the algorithm are increased. The efficiency of the proposed algorithm is proven by the evaluation conducted on the synthetic dataset compared to the other algorithms implementing the system. Varying several parameters, the proposed system in this paper is proven to be proper to adopt in the real world. Furthermore, the paper suggests two platform systems as future works based on the pattern recognition and CCH algorithm in data-collecting for more optimal path recommendation.

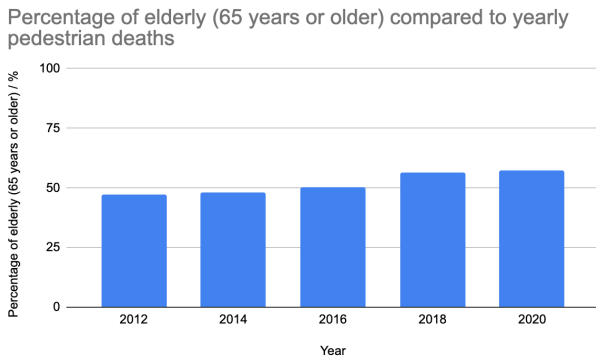
Keywords: *Navigation system, Informatics, Database, Skyline query, Recommendation*

1. Introduction

In a society where people try to spend less time and energy, especially when traveling, finding the best route has become fundamental in modern navigation systems. Commonly used navigations, such as Naver Map and Kakao Map, prioritize finding the route that takes the least time and has the shortest distance. Users can choose the means of transportation and the proposed optimal routes are different. For example, if a user chooses a personal vehicle as a means of transportation, the map proposes not only the shortest distance and time, but also the path consisting of the roads that the user frequently passes. Public transportation provides a route for the user to transmit in the smallest time possible, or a route for the user to walk less.

As the navigation system began to be developed, programs for specific targets were also developed. Maps for the blind have developed greatly, and voiceovers that guide the blind safely to their destinations help them avoid obstacles. It was developed because visually impaired people cannot recognize surrounding objects and lack facilities to assist them even in outdoor environments.[1]

One thing society forgets is that old people also apply to previous sentences. They have mobility, speed, small size, and a degraded ability to detect a reactor or motion as well as sound. In 2025, Korea will be classified as a "super-aged" society with more than 20% of the population aged 65 or older[2]. At the same time, the proportion of the elderly in pedestrian deaths is increasing.



<figure 1>

As shown in <figure 1>, Koreans aged 65 or older accounted for 57% of pedestrian deaths in 2020. The

death rate of pedestrians in Korea is three times higher than the average of the Organization for Economic Cooperation and Development (OECD) and the elderly account for more than half of the deaths, so elderly pedestrian accidents are a big burden to Korea. However, navigation is not suitable for the elderly. This research starts from here.

The Contribution points in the paper are: first, we studied the criteria and the factors that the system should consider for the elderly in finding the way and defined score function based on it. Second, by adopting Skyline Query, the performance and utility of the algorithm were increased.

The design of the paper is followed. The second chapter introduces the important factors proposed in the score function of this paper and the score database using them. The third chapter describes the baseline algorithm and the fourth chapter introduces an improved algorithm that introduces the skyline query. The proposed method is explained in the fifth chapter, leading to an analysis of experimental results in the sixth chapter. The seventh chapter concludes the paper while presenting future works related to the research. References used are mentioned in the eighth chapter.

2. Score Function

2.1 Factor Research

Reasons for elderly pedestrian deaths can be split into three different categories: on roads, on crosswalks, and on sidewalks.

First of all, on car roads, 48.4% of deaths of elderly pedestrians deaths occur due to their violation of road laws. Commonly, this is caused by jaywalking. Jaywalking is the act of crossing a road when cars are still driving by. It is often done when the pedestrian can not find an obvious, quick path. This is especially dangerous for the elderly as they have low agility and can not react quickly nor detect danger quickly. To prevent jaywalking, seniors should use navigation systems. However, it is not easy for them to operate the applications and they can not walk safely while staring down at their phone. Moreover, normal navigation does not lead them to the safest roads. The way to solve this problem is to provide navigation

systems suited to the elderly and provide them with safe, quick paths.

Secondly, 60% of deaths while crossing the crosswalks are among seniors.[4] It is due to the fact that the elderly do not have enough time to cross the road and has trouble seeing how much time is left to cross. The best solution is for the government to increase the time for the crosswalk signs. Another way navigation systems could solve this is by checking whether the time is plentiful depending on their average speed or helping them avoid crosswalks.

Last but not least, on sidewalks, there are accidents that happen while walking. The cause of the second-highest percentage of deaths of the elderly is known to be falling down.[5] 28.5% fell down due to slippery ground, and 25% due to steep slopes and uneven sidewalks. The rest are sprained ankles or tripping over sidewalks. These problems can be mitigated by navigation systems that avoid these impediments.

2.2 Score Function

Considering the different factors the elderly have to avoid when traveling, this paper introduces a new navigation system for the elderly. A score function is implemented, with higher scores given to paths suited for the seniors and lower scores for those not. The navigation needs a database for factor scores. By defining a new score function based on it, the navigation system will be able to find the optimal path suited for the old. The scale and priority will be able to be changed by user selection.

Example 1. *<table 1>* is an example of the factors and scores for each factor.

Factors	Priority	Neg/pos example (scale)
Gradient of slope	1	- 1 * gradient
Bumpiness of road	5	- 0.7
Crosswalks without traffic lights	2	- 0.9

Time left for crosswalks	4	- 0.1 * time left
Pedestrian road (no cars)	3	- 0.8
Narrow road	7	- 0.5
(Nighttime only) Street lamp	6	+ 0.7

<table 1>

The first thing the system should consider is the priorities. These attributes do not have the same label. And then, the users score the amount of score. If the value is negative, it means the factor plays a negative role in finding the path for targets. Otherwise, the exact opposite happens.

3. Baseline Algorithms

3.1 Baseline Methods

Two basic methods for the proposed system are: brute force and greedy algorithms. In section A, the previous approach is explained. The Brute Force and the greedy algorithm are followed in section B and section C.

3.1.1 Problem Definition

In the current navigation system, the only factor it considers is the shortest path between two POIs:

$$MIN \text{ dist}(S, D) \quad \dots \text{formula 1}$$

, where S for the start location, D for the destination, and $dist()$ for the distance in a road network RN . RN is represented with node set N and edge set E .

The proposed system adopts a score function below instead of *formula 1*:

$$S(p) = \sum_{e \in EV} T(e) \quad \dots \text{formula 2}$$

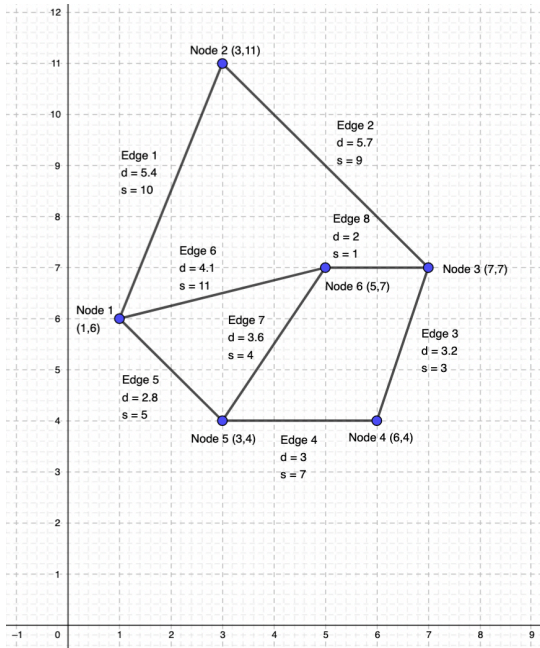
, where EV for the set of edges visited, p is the selected path, $S()$ for the total score and for the score which is represented as:

$$scale / (priority \times dist(e)) \quad \dots \text{formula 3}$$

Example 2. Consider the road network RN in <figure 2>. $|N| = 6$, $|E| = 8$, each $dist(e)$ noted as d , and each $T(e)$ noted as s .

Suppose the user sets n_1 as the starting location and n_3 for the final destination. In the previous approach, the system returns path $p = \{n_1, n_6, n_3\}$ and the total distance be $4.1 + 2 = 6.1$, and $S(p) = 11 + 1 = 12$.

In the proposed system, as $T()$ will be considered, $p' = \{n_1, n_2, n_3\}$, and $S(p') = 10 + 9 = 19$. Compared to p , p' returns 5 units longer distance. This means that the previous approach does not return the best route for the targets.



<figure 2>

3.1.2 Brute Force

Brute force algorithms are accurate as every possible path from the starting node to the destination is calculated and compared. However, a big disadvantage is that a huge amount of time is taken to run the code. This is a serious problem if implemented into the navigation system as the system will not be quick enough and lagging.

	Distance	Score	Nodes
--	----------	-------	-------

Best Distance	$4.1 + 2 = 6.1$	$11 + 1 = 12$	$1 \rightarrow 6 \rightarrow 3$
Best Score	$5.4 + 5.7 = 11.1$	$10 + 9 = 19$	$1 \rightarrow 2 \rightarrow 3$

<table 2>

<table 2> above is the answer of brute force algorithm when it comes to <figure 2>. Since the explanation is already in **Example 2**, I may pass one.

The time complexity of the brute force algorithm is $TC(m, n) = O(|N|^2 + m)$. The total number of path is $\sum_{i=1}^{|N|-2} COMBIN(|N|, i) \geq |N|^2$. m is for comparison.

Considering that the road networks in the real world contain a lot of nodes, This method cannot be applied in the performance aspect.

3.1.3 Greedy Algorithm

Greedy algorithms, which select the best option available at the moment, are inaccurate. Although it may take less time than brute force algorithms, as it does not operate exhaustively on all the data, makes commitments to certain choices too early, and is unable to consider future possibilities. This prevents the algorithm from finding the optimal solution. As this does not meet the program's intention to find the optimal path, it can not be utilized in the new navigation system.

	Distance	Score	Nodes	Edges
Best Distance	$2.8 + 3 + 3.2 = 9$	$5 + 7 + 3 = 15$	$1 \rightarrow 5 \rightarrow 4 \rightarrow 3$	$5 \rightarrow 4 \rightarrow 3$
Best Score	$4.1 + 2 = 6.1$	$11 + 1 = 12$	$1 \rightarrow 6 \rightarrow 3$	$6 \rightarrow 8$

<table 3>

<table 3> above is the answer of brute force algorithm when it comes to <figure 2>. In the aspect of the best distance, the greedy algorithm first selects the minimum distance edge among the edges which are connected to the start point. When it comes to the aspect of the best score, the greedy algorithm first selects the one with the best score. The greedy algorithm surely returns the answer in time complexity no more than brute force.

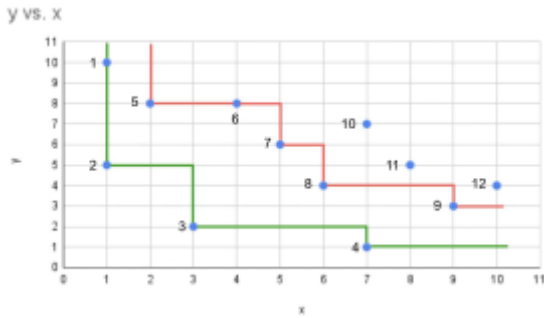
However, this method has an innate problem: it doesn't always return the correct answer. As you can see in <table 2> and <table 3>, the algorithm neither returns the minimum distance route nor the best-scored route.

The trade-off of time and accuracy occurs quite often. However, considering the real-world issue, the pressure of time isn't a burden since the features of the roads are not pliant. Once it calculates all the values of the feature, it barely changes. Thus, in this research, I implemented the greedy algorithm to show how much the system should sacrifice correctness when applying a greedy algorithm because of the time complexity.

4. Skyline

4.1 Skyline

Skyline, known as Maxima in computational geometry or Pareto in the business management field, returns the optimal points that can not be dominated by the rest of the points in the dataset[8].



<figure 3>

Here, 'dominates' refers to the points that are the most 'optimal' compared to the rest of the points – in graphs, they are the ones closest to the origin of the graph. As it gets closer to the origin, it increasingly satisfies the user's preferences.

Example 3. Considering two points p3 and p8 in <figure 3>, p8 is eliminated as p3 suits the user's preference more.

The distance between the origin of the graph and p3 is:

$$\sqrt{3^2 + 2^2} = \sqrt{13}.$$

The distance between p3's x-coordinate and the origin is 3, and the distance between the y-coordinate and the origin is 2. This makes up a total of 5.

The distance between the origin of the graph and p8 is :

$$\sqrt{6^2 + 4^2} = \sqrt{52}.$$

The distance between p8's x-coordinate and the origin is 6, and the distance between the y-coordinate and the origin is 4. This makes up a total of 10.

As p3 is closer in distance than p8 from the origin, p8 is eliminated and p3 is considered 'optimal.'

Thus, the first skyline layer, which contains the points that meet the user's preferences is {p1, p2, p3, p4}. The other points are all less suited to the user's preferences.

The decision-making of the skyline query is useful as it makes effective and accurate decisions, considering every factor of the user's selected preferences. Compared to brute force which takes longer and greedy algorithms where inaccurate paths may be chosen, the skyline query is much more applicable when finding the optimal path for the navigation system.

4.2 Multiple Skyline Layers

Multiple skyline layers can be defined as the first k skyline layers. The first skyline layer is the subset of points closest to the origin, and the second is the skyline layer closest to the origin if the first set of subsets of the first skyline layer is not considered.

Example 4.

Considering fig 3. The green layer is the first skyline layer, the connection of points closest to the origin of the graph. In other words, it is the optimal path. The red layer is the second skyline later, which is the connection of points closest to the origin when the first skyline layer is not considered.

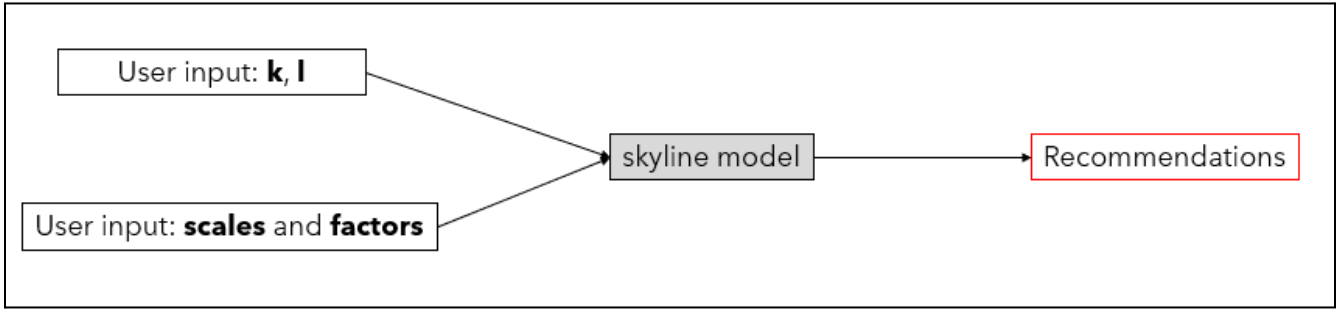
Thus the layers in <figure 3> are as followed:

Layer 1 = {1(1,10), 2(1,5), 3(3,2), 4(7,1)}

Layer 2 = {5(2,8), 6(4,8), 7(5,6), 8(6,4), 9(9,3)}

Layer 2 = {10. 11. 12}

It should be noted that just because it exists in the latter layer does not necessarily dominate the points of the front layer. For example, point 3 and point 5 do not have a controlling relationship with each other.



<figure 4>

5. Proposed method

To ensure the user's preference, it may simply apply top-query to brute force to return the answer. However, in the problem covered in this paper, the features of each point span several dimensions, and each attribute has a different degree of preference. The previously introduced skyline query can supplement this part and suggest to people a certain number of 'things that are not worse than other points' rather than the correct answer consisting of k absolute values.

The proposed system takes two kinds of input: one which controls the return answer; the other one for the scoring system. In <figure 4>, with precalculated skyline model, those input proposes proper recommendations of routes to users. Value k represents how many recommendations the user wants. Value l represents the maximum layer. Denote that the points on layer l are dominated by at least $l-1$ points[9].

6. Experimental Results

6.1 Synthetic data

To evaluate the proposed system, I generated a synthetic dataset. The number of nodes and edges, the scores of each edge, and the coordination of each node are randomly generated.

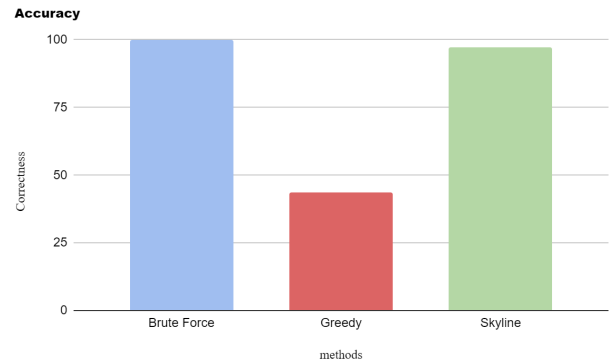
node	100	200	300	400	500	600	700	800	900
edge	1:1.1	1:1.3	1:1.5	1:1.7	1:1.9	1:2			

<table 4>

The number of nodes varies from 100 to 900, the ratio of node : edge varies from 1:1.1 to 1:2. The number of nodes is fixed to 500 when varying the number of

edges. The number of edges is fixed to $1.5 \times |nodes|$ when varying the number of edges.

6.2 Accuracy



<figure 5>

As shown in <figure 5>, the accuracy varies between the three methods. The greedy algorithm has the lowest accuracy, brute force the highest, and the Skyline method is close but still is less accurate compared to brute force. The reason greedy algorithms have lower accuracy is the fact that the best selection at a certain point is not necessarily the optimal selection as a whole, but greedy algorithms select the best option at each node. On the contrary, as brute force algorithms process every single possible route, it will always be accurate in finding the optimal path. The skyline method is in the majority of the time accurate, but not always as it is based on the domination of points rather than comparing all the possible paths.

6.3 Time Complexity Comparison

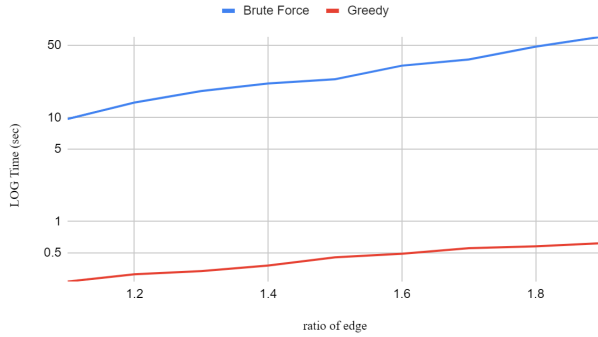
6.3.1 Comparison: BF vs. Greedy

Time Complexity Comparison: BF vs G



<figure 6>

Time Complexity Comparison: BF vs G

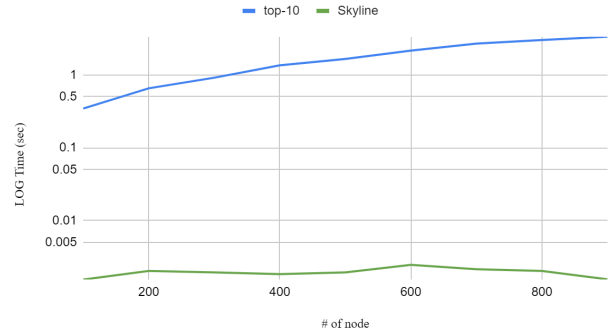


<figure 7>

Both <figure 6> and <figure 7> show the comparison between the time complexity when the brute force algorithm is implemented and when the greedy algorithm is implemented, each based on the number of nodes or the ratio of edges. The two graphs clearly show that the greedy algorithm has remarkably lower time complexity. This is because greedy algorithms only search for the optimal path once, but brute-force algorithms search through every possible path.

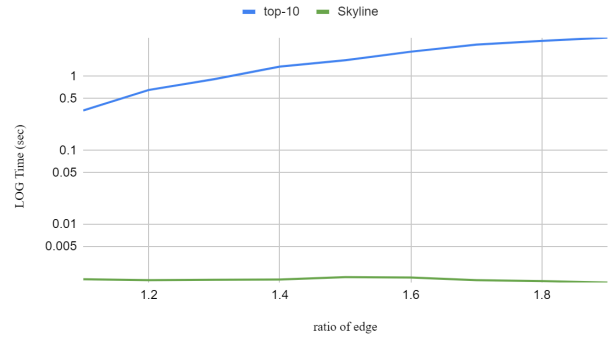
6.3.2 Comparison: top-10 BF vs. Skyline

Time Complexity Comparison: BF+top-10 VS Skyline



<figure 8>

Time Complexity Comparison: BF+top-10 VS Skyline



<figure 9>

Both <figure 8> and <figure 9> show the comparison between the time complexity when the brute force algorithm is implemented and when the skyline method is implemented, each based on the number of nodes or the ratio of edges. The two graphs clearly show that the skyline method has remarkably lower time complexity. This is because skyline methods search for the optimal path once, but brute force algorithms search through every possible path.

7. Conclusion and Future Work

The Skyline Query was selected for the navigation system due to several reasons. The brute force method was first considered because of its perfect accuracy; however, a new algorithm was needed as its time complexity was too great. It was then replaced with the greedy algorithm, which had trade-offs between the high accuracy of the brute force and its low time complexity. Thus, the skyline method was adopted as it not only had both accuracy and low time complexity, but it could also consider every factor at once.